Design note: source schema for the Olist subset

Counts are for the loaded subset (the first 10,000 orders by `order_id`) unless marked "source", which means the full CSV files.

1. Entities and grain

`customers` has one row per `customer_id`, which Olist issues once per order; it holds the buyer's lasting identifier (`customer_unique_id`) and the city and state for that order. `sellers`, `products`, and `orders` have one row per seller, listed product, and order. `order_items` has one row per item of an order, identified by the order and the item's position in it; it names the product, the seller, and the price and freight charged for that item.

2. Functional dependencies that hold

These follow from how Olist defines each identifier; we used the data only to look for counterexamples and found none.

- customers: customer_id → customer_unique_id, customer_city, customer_state
- sellers: seller_id → seller_city, seller_state
- products: product_id → product_category_name, product_weight_g, product_length_cm, product_height_cm, product_width_cm
- orders: order_id → customer_id, order_status, order_purchase_timestamp, order_approved_at, order_delivered_carrier_date, order_delivered_customer_date, order_estimated_delivery_date
- orders: customer_id → order_id. A `customer_id` belongs to exactly one order, so {customer_id} is a second candidate key of `orders`, and we declare it UNIQUE.
- order_items: {order_id, order_item_id} → product_id, seller_id, price, freight_value

3. Dependencies that do not hold

Dependency	Counterexamples in the subset
product_id → price	634 products sell at more than one price
product_id → seller_id	118 products are sold by more than one seller
order_id → product_id; order_id → seller_id	317 orders hold more than one product; 113 involve more than one seller
customer_unique_id → customer_city, customer_state	2 people appear with more than one city, 1 with more than one state
customer_city → customer_state	23 customer city names occur in more than one state

So price and seller sit on the item, location sits on the `customer_id` row and not on the person, and state is stored beside city.

4. 3NF argument

A table is in 3NF if, for every nontrivial dependency X → A that holds in it, X is a superkey or A is part of some candidate key. Every column holds a single value, so the tables are in 1NF.

- **customers.** The only candidate key is {customer_id}, and every dependency in section 2 has it on the left. No other determinant exists: customer_unique_id does not determine city or state, and city does not determine state.
- **sellers.** The only candidate key is {seller_id}. City does not determine state.
- **products.** The only candidate key is {product_id}. A category does not fix a weight or a dimension, and the measurements do not fix each other.
- **orders.** The candidate keys are {order_id} and {customer_id}. Every dependency that holds has one of them on the left. Status does not determine a timestamp, and no timestamp determines another.
- **order_items.** The only candidate key is {order_id, order_item_id}. There is no partial dependency: order_id alone determines neither product, seller, price, nor freight, and order_item_id alone is a position from 1 to 8 that repeats across orders. No non-key column determines another: product_id determines neither price nor seller_id. Section 5 covers the one pattern that resembles such a dependency.

5. Decomposition decisions

- **Zip prefix left out.** The zip prefix determines the state on all 99,441 source customer rows, so keeping it would create the transitive dependency customer_id → zip prefix → state. The alternative is a location table keyed by zip prefix. We did not take it: no planned question needs zip detail and the geolocation file is out of scope.
- **shipping_limit_date left out.** It is constant per (order_id, seller_id) across the source, while 336 source orders carry more than one value, so it depends on neither the order alone nor the full item key. The alternative is an order-seller table. We did not take it: no planned question uses the date.
- **Price and freight kept on the item.** Repeated units of one product from one seller in one order always share price and freight in the source: 7,088 such groups among 112,650 rows, no exceptions. As a rule, that would be {order_id, product_id, seller_id} → price, freight_value with a left side that is not a superkey, and the alternative would be an order line keyed by those three columns with a quantity. We did not take it: Olist documents both values per item and nothing states that two units must cost the same, so we treat the equality as a pattern in this extract, not a rule.
- **No person table.** `customer_unique_id` identifies the person, but no column depends on the person alone, since city and state can change between one person's orders. The table would hold only its key.
- **No category table.** The category name has no dependent columns in our scope, so a lookup table would hold a single column.
- **customers kept apart from orders.** A one-to-one relationship could be merged into one table. We keep buyer and order as separate entities linked by a UNIQUE foreign key, because later analysis treats the customer as its own dimension.

6. Constraints, exceptions, and rerun

Enforced. A primary key on every table; foreign keys from `orders` to `customers` and from `order_items` to `orders`, `products`, and `sellers`, with the default delete behaviour, so a referenced parent cannot be removed; NOT NULL on every required column; UNIQUE on `orders.customer_id`. CHECK constraints cover identifier and state format, the eight order statuses, positive measurements, price above 0, freight at or above 0, `order_item_id` at or above 1, and approval, customer delivery, and estimated delivery not earlier than purchase.

Left unenforced. A constraint against any of these patterns would reject real rows, so the schema allows them and the verification script reports each count.

Pattern	Count	Why it is allowed
Delivered order with no customer delivery date; with no approval timestamp	1; 1	Status and dates are recorded separately; the extract does not say which is wrong.
Canceled order that has a customer delivery date	1	An order can be canceled after it arrives.
Carrier handover recorded before purchase; customer delivery before carrier handover	20; 1	We cannot tell an entry error from a real event, so the carrier date has no ordering CHECK.
Product with no category	103	The source gives none; the column is nullable.
Order item with zero freight	40	Zero is a valid charge; the CHECK is freight at or above 0.
customer_unique_id under more than one customer_id	34	Returning buyers; this is why the column is not UNIQUE.

That every customer has an order cannot be declared without a circular foreign key, so a verification query confirms it (0 customers without an order). Orders without items are valid: 76 have none (55 unavailable, 21 canceled).

Rerun. The load runs in one transaction, parents first, and any error rolls back all five tables. Each row is written with INSERT ... ON CONFLICT (primary key) DO UPDATE, applied only where the stored row differs from the incoming row. A second run on unchanged input inserts 0 and updates 0 rows in every table, and the fingerprints and the price and freight sums are identical between runs. The limit: a row that disappears from the input is not deleted from the database; to remove rows, recreate the database and load again.